

Slides Activity 2:

Repeated sampling from the data generating mechanism

Table of contents

One Thousand Random Samples from the Data Generating Mechanism	1
Before you begin	2
Step 1. Outline the specific aims.	2
Step 2. Determine the appropriate data generating mechanism.	2
Helper functions	2
Function for the DGM	3
Set the seed	4
Generate the “truth”	4
Resample 1,000 times	4
Step 3. Define the estimand of the analysis.	5
Step 4. Identify the methods to be implemented in the study.	5
Add IPT weights to the samples	6
Sample-specific analysis	6
Step 5: Performance measures	7
Sample-specific set-up	7
Estimate performance measures	8
Plots	8
When things go wrong	11

One Thousand Random Samples from the Data Generating Mechanism

We want to create 1,000 random samples of 500 people from our data generating mechanism (DGM). This allows us to understand the sampling properties of our estimators and compare approaches for estimating standard errors.

To do this, we need to build a function that samples from the DGM. Luckily, we can adapt the code we wrote for one sample in Activity 1.

Before you begin

Install the required packages.

```
library(dplyr)
library(purrr)
library(ggplot2)
library(sandwich)
```

Step 1. Outline the specific aims.

Context: We want to estimate the effect of prenatal vitamin supplementation on the risk of preterm birth. However, we are worried about confounding by maternal smoking, which is not measured in our data. We also want to understand the implications for different approaches to standard error estimation.

Simulation Objectives:

1. Quantify the magnitude and direction of bias in our estimate of the effect of prenatal vitamins on the risk of preterm birth under scenarios where maternal smoking decreases the use of prenatal vitamins but increases the risk of preterm birth (Activity 1).
2. Compare the 95% confidence interval coverage for the risk difference using different approaches to standard error estimation, including model-based SEs and sandwich SEs (Activity 2).

Step 2. Determine the appropriate data generating mechanism.

Helper functions

First, create the “helper” functions we will use throughout our simulation.

```
# The expit function is the inverse of the log-odds. It takes in a log-odds (x),
# applies the inverse-logit function and returns a probability
expit <- function(x) {
  return(exp(x) / (1 + exp(x)))
}
```

```

# We also include the balancing intercept function in case we want to
# change around any parameters and it needs to be re-derived
derive_intercept <- function(py, prob_prenatal, prob_smoke,
                             or_smoke, or_prenatal){

  # marginal probability of treatment
  prob_prenatal <- prob_prenatal[1] * (1-prob_smoke) + prob_prenatal[2] * prob_smoke

  -log(1/py - 1) - or_prenatal*prob_prenatal - or_smoke*prob_smoke
}

```

Function for the DGM

Create a function for the DGM that we can randomly sample from. The code is near-identical to the code from the one-random-sample example in Activity 1, but adds in a derivation for the intercept so we can change parameters without having to re-derive it by hand each time.

```

dgm <- function(simid = 0,
                n = 500,
                prob_smoke = 0.2,
                prob_prenatal = c(0.85, 0.65),
                or_prenatal = log(0.9),
                or_smoke = log(3),
                prob_preterm = 0.1){

  # Dynamically derive the intercept term
  y_intercept <- derive_intercept(py = prob_preterm,
                                   prob_prenatal = prob_prenatal,
                                   prob_smoke = prob_smoke,
                                   or_smoke = or_smoke,
                                   or_prenatal = or_prenatal)

  dplyr::tibble(
    # Simulation number
    simid = simid,

    # Create n participants
    id = 1:n,

    # Randomly select smoking
    smoke = rbinom(n = n, size = 1, prob = prob_smoke),

```

```

# Generate potential outcome under no treatment
preterm_x0 = rbinom(n = n, size = 1,
                    prob = expit(y_intercept + (or_smoke * smoke))),

# Generate potential outcome under treatment
preterm_x1 = rbinom(n = n, size = 1,
                    prob = expit(y_intercept + (or_smoke * smoke) + or_prenatal)),

# Generate observed treatment conditional on smoking
p_prenatal = prob_prenatal[smoke + 1], # probability of prenatal based on smoking
p_prenatal_margin = prob_prenatal[1]*(1-prob_smoke) + prob_prenatal[2]*(prob_smoke),
prenatal = rbinom(n = n, size = 1, prob = p_prenatal),

# Assign actual outcome conditional on observed treatment
preterm = preterm_x0*(1-prenatal) + preterm_x1*(prenatal)
)
}

```

Set the seed

ALWAYS before any random number generation!

```

# Set the random seed. It's good to use a relatively random starting number, not just 1234 or 5678
set.seed(57846)

```

Generate the “truth”

First, create a large re-sampled dataset from which to derive the truth. By doing this after setting the seed, we ensure that we use the same random state start and thus do not overlap with random states from the “observed” data. See Morris et al. (2019) for additional detail on this.

```

# Random sample for estimating the truth
truth <- dgm(n = 10000000, prob_prenatal = c(0.85, 0.65), or_smoke = log(3))

```

Resample 1,000 times

Now we want to re-sample from this DGM 1,000 times to understand the sampling behavior of our estimators. Note that we are *not* resetting the seed with each iteration. See Morris et al. (2019) for additional detail.

```
# Randomly sample 1,000 datasets of 500 people for the analysis
data1000 <- purrr::map(.x = 1:1000, dgm, n = 500, prob_prenatal = c(0.85, 0.65), or_smoke = 1)
```

Step 3. Define the estimand of the analysis.

The slides illustrate how to derive the truth mathematically. However, this is sometimes challenging to do but can be estimated from the DGM with a large n . So, we illustrate how to do that below.

```
# Estimate the "true" RD using potential outcomes -- no complex analysis needed!
true1 <- mean(truth$preterm_x1)
true0 <- mean(truth$preterm_x0)
true_rd <- true1 - true0

dplyr::tibble(
  Estimator = "Truth",
  `Risk prenatal` = true1,
  `Risk, no prenatal` = true0,
  `Risk Diff` = true_rd
) |>
dplyr::mutate(
  dplyr::across(-Estimator, \(x) round(100*x, 1))
)
```

```
# A tibble: 1 x 4
  Estimator `Risk prenatal` `Risk, no prenatal` `Risk Diff`
  <chr>          <dbl>          <dbl>          <dbl>
1 Truth          10.6          11.6           -1
```

Step 4. Identify the methods to be implemented in the study.

We are comparing two estimators:

1. Unweighted risk difference
2. IPT weighted risk difference (known weights)

Within the weighted risk difference, we are further comparing two approaches for standard error estimation:

1. Model-based SEs
2. Sandwich estimator for SEs

Add IPT weights to the samples

We add the “known” treatment weight to each sample — known because we set the true treatment-assignment probabilities for the simulation. In a real observational study we wouldn’t know these; we’d have to estimate instead.

```
data1000 <- purrr::map(data1000,
  function(x){
    x |>
    dplyr::mutate(
      # Known weight: numerator is the marginal probability
      # of treatment, denominator is the probability of the
      # observed treatment given smoking
      ipw = if_else(prenatal == 1,
                    p_prenatal_margin/p_prenatal,
                    p_prenatal_margin/(1-p_prenatal))
    )
  }
)
```

Sample-specific analysis

Within each sample, we want to estimate the unadjusted risk difference and the weighted risk difference and their associated model-based standard errors. Additionally, for the *weighted* risk difference we will estimate the robust sandwich standard errors to account for the weights.

```
analyze_each_sim <- function(x){
  unadj <- glm(preterm ~ prenatal, data = x, family = binomial(link = "identity"))
  adj_known <- glm(preterm ~ prenatal, data = x,
    weights = ipw, family = quasibinomial(link = "identity"))

  dplyr::tibble(
    estimator = c("Unadjusted", "Adjusted (Naive SE)", "Adjusted (Robust SE)"),
    # risk under no treatment
    r0 = c(coef(unadj)[1], coef(adj_known)[1], coef(adj_known)[1]),
    # extract risk difference from the models
    rd = c(coef(unadj)[2], coef(adj_known)[2], coef(adj_known)[2]),
    # risk under treatment
    r1 = rd + r0,

    # extract the standard errors from the models
    se = c(sqrt(diag(vcov(unadj))))[2],
```

```

      sqrt(diag(vcov(adj_known))) [2],
      sqrt(diag(sandwich::vcovHC(adj_known))) [2])
    )
  }

# loop over each sample and apply the analysis
rds_ses <- purrr::map_dfr(
  data1000,
  analyze_each_sim
)

```

Step 5: Performance measures

Sample-specific set-up

To estimate bias and coverage, we need to do an additional data manipulation to the results from each simulation sample.

95% confidence interval coverage is the proportion of times the true risk difference is captured within the 95% confidence interval. Within each sample, we create a binary variable that is 1 if the confidence interval from that sample contains the true risk difference and 0 if it does not:

$$I(\hat{\theta} - 1.96 \times \hat{SE}(\hat{\theta}) \leq \theta \leq \hat{\theta} + 1.96 \times \hat{SE}(\hat{\theta}))$$

where $I()$ is the indicator function (= 1 if expression in parenthesis is true, = 0 if false)

Bias is mean error across simulations. Within each sample, we calculate error as:

$$\hat{\theta} - \theta$$

```

rds_ses <- rds_ses |>
  dplyr::mutate(
    # calculate error (estimated rd - true rd)
    error = rd - true_rd,
    # does the true rd fall within the estimated 95% CI?
    model_coverage = (true_rd >= (rd - 1.96*se)) & (true_rd <= (rd + 1.96*se))
  )

```

Estimate performance measures

Summary statistics of the simulation-specific values we already estimated

```
measures <- rds_ses |>
  dplyr::summarize(
    exp_r1 = mean(r1),
    exp_r0 = mean(r0),
    exp_rd = mean(rd),
    bias = mean(error),
    ese = sd(rd),
    ase = sqrt(mean(se^2)),
    model_coverage = mean(model_coverage),
    .by = estimator
  )

measures |>
  dplyr::mutate(
    dplyr::across(-estimator, \(x) round(100*x, 1))
  ) |>
  kableExtra::kable(col.names = c("Estimator", "Risk, prenatal", "Risk, no prenatal", "Risk I
```

Estimator	Risk, prenatal	Risk, no prenatal	Risk Diff	Bias	ESE	ASE	95% CI Coverage
Unadjusted	10.1	14.1	-4.1	-3.1	4.0	3.9	90.0
Adjusted (Naive SE)	10.6	11.8	-1.3	-0.3	3.8	2.8	85.3
Adjusted (Robust SE)	10.6	11.8	-1.3	-0.3	3.8	3.7	93.5

Plots

Below are some ideas for presenting simulation results.

Well, the first one is more just to get a feel for what the sampling distribution looks like.

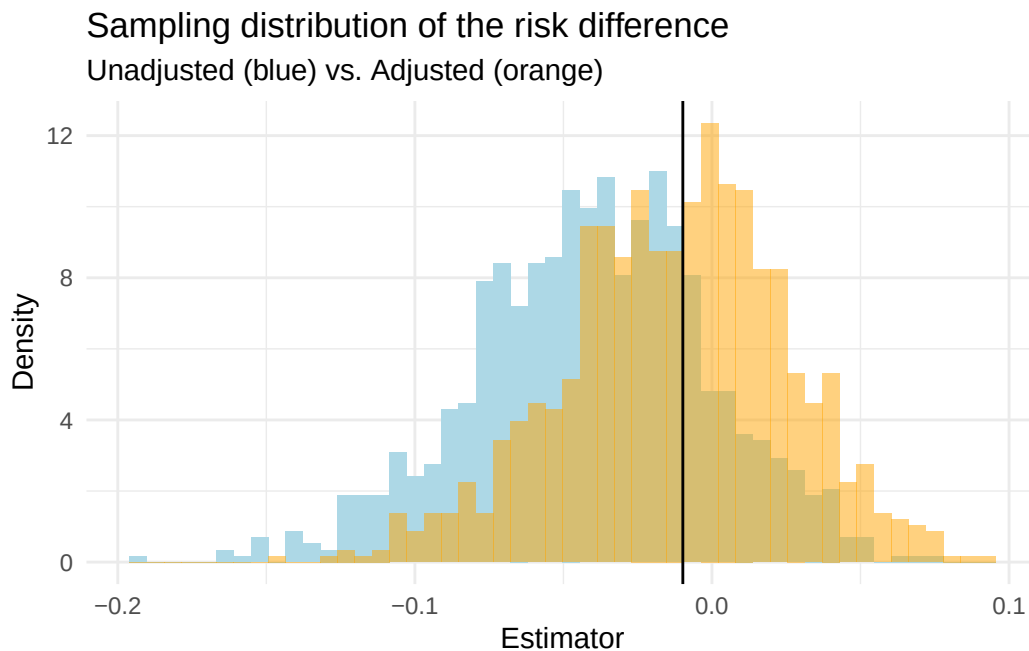
```
rds_ses |>
  dplyr::filter(estimator == "Unadjusted") |>
  ggplot(aes(rd, after_stat(density))) +
  geom_histogram(bins = 50, fill = "lightblue") +
  geom_histogram(data = rds_ses[rds_ses$estimator == "Adjusted (Naive SE)",],
```



```

      bins = 50, fill = "orange", alpha = 0.5) +
theme_minimal() +
geom_vline(xintercept = true_rd) +
labs(
  title = "Sampling distribution of the risk difference",
  subtitle = "Unadjusted (blue) vs. Adjusted (orange)",
  x = "Estimator",
  y = "Density"
)

```

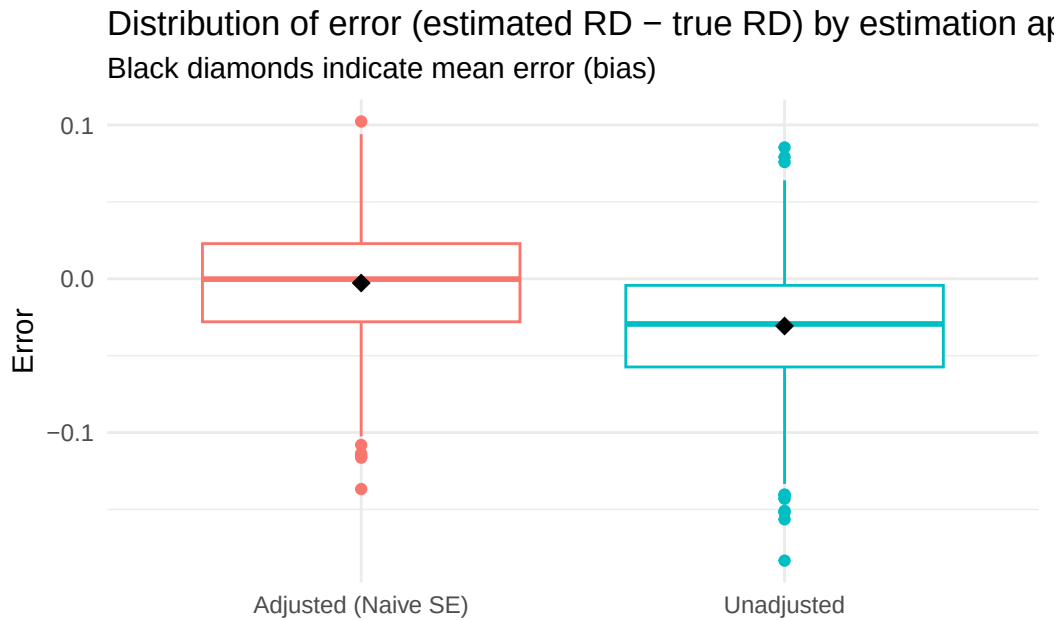


The boxplot is a nice way to visualize bias and spread all in one figure. This shows bias and ESE – so doesn't give a good sense of how the standard error estimators are doing.

```

rds_ses |>
  dplyr::filter(estimator != "Adjusted (Robust SE)") |>
  ggplot(aes(x = estimator, y = error, color = estimator)) +
  geom_boxplot(show.legend = FALSE) +
  stat_summary(fun = mean, geom = "point", shape = 18, size = 3, color = "black") +
  theme_minimal() +
  labs(x = "",
       y = "Error",
       title = "Distribution of error (estimated RD - true RD) by estimation approach",
       subtitle = "Black diamonds indicate mean error (bias)")

```

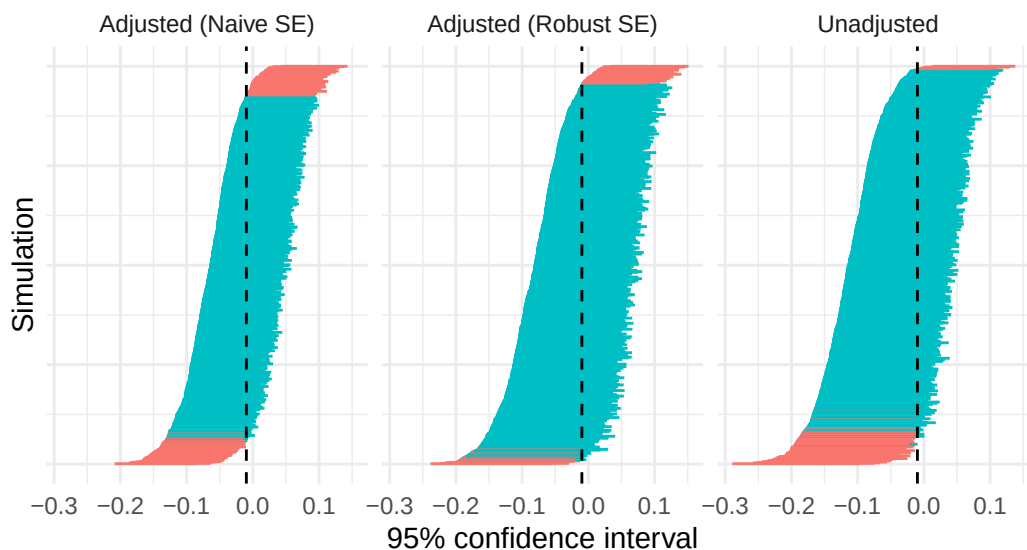


Below shows how coverage is performing in each simulation. Unadjusted is doing the worst (the most red/pink), whereas the adjusted with robust SE is doing the best (the most blue).

```
rds_ses |>
  dplyr::mutate(
    lcl = rd - 1.96*se,
    ucl = rd + 1.96*se
  ) |>
  dplyr::arrange(lcl) |>
  dplyr::mutate(id = row_number(), .by = estimator) |>
  ggplot(aes(y = id, x = lcl, xend = ucl, color = model_coverage)) +
  geom_segment(show.legend = FALSE) +
  geom_vline(xintercept = true_rd, linetype = "dashed") +
  facet_wrap(~estimator) +
  theme_minimal() +
  theme(axis.text.y = element_blank()) +
  labs(y = "Simulation", x = "95% confidence interval",
       color = "",
       title = "95% confidence interval coverage across simulations",
       subtitle = "Blue = interval contains the true risk difference")
```

95% confidence interval coverage across simulations

Blue = interval contains the true risk difference



When things go wrong

If we have parameters close to the edge of the parameter space (e.g., rare exposure, rare outcome), we can run into issues with model convergence in some of the samples.

We can set up our simulation code defensively to collect when a simulation fails and keep going, rather than failing and stopping the program altogether.

For example, let's make the comparator more rare (i.e., more people taking prenatal vitamins)

```
data_error <- purrr::map(
  .x = 1:1000,
  dgm,
  n = 500,
  prob_prenatal = c(0.95, 0.85),
  prob_smoke = 0.1)

rds_ses_error <- purrr::map(
  data_error,
  function(x){
    x <- x |>
    dplyr::mutate(
      # Known weight: numerator is the marginal probability
```

```

# of treatment, denominator is the probability of the
# observed treatment given smoking
ipw = if_else(prenatal == 1,
              p_prenatal_margin/p_prenatal,
              p_prenatal_margin/(1-p_prenatal))
)

tryCatch(
  analyze_each_sim(x), # same function as above, just wrapped in a tryCatch
  error = function(e){
    return(paste0("Error in simulation: ", e$message))
  }
)
}
)

idx <- which(purrr::map_lgl(rds_ses_error, is.character))

```

The simulation failed at the 179, 340, 487, 494, 499, 647, 692, 724, 867, 871, 937, 950 indexes. We can now see why our simulation failed because we saved the error messages. When estimating our performance measures or calculating Monte-Carlo error, we will need to take into account our actual successful simulations is smaller.